

Product Requirements Document

1. Product Overview

Product Name

Internship Management Platform

Problem Statement

Universities often manage internship opportunities, student applications, advisor approvals, and internship progress tracking across spreadsheets, forms, chat messages, and manual follow-up. This creates slow approvals, duplicate work, unclear status visibility, and inconsistent records.

Target Users

- Students looking for and applying to internship opportunities
- Academic advisors who review and approve internship applications
- Program administrators who manage placement data and monitor workflow status

Product Goal

Provide a single web platform where students can browse placements, submit applications, track approval status, and submit progress updates while advisors can review and approve applications in a simple workflow that is realistic for a one-semester student project.

2. Scope

In Scope

- Opportunity explorer for internship placements
- Placement detail page
- Student application submission
- Student application status tracking
- Advisor approval or rejection workflow
- Student internship progress update submission
- Basic role-based access for students, advisors, and administrators
- Internship status change event publishing
- Simple skill matching support with AI-assisted matching as an optional enhancement and tag-based fallback

Out of Scope

- Full company portal for external employers
- Advanced analytics dashboards
- Real-time chat or messaging
- Complex notification center
- Native mobile application

- Payroll, contracts, or legal document workflow
 - Multi-university tenancy
-

3. User Roles

Student

- Browse internship placements
- View placement details
- Submit an application
- View own application history and status
- Submit progress updates during the internship

Advisor

- View assigned or relevant student applications
- Review student application details
- Approve or reject applications
- Review submitted progress updates when needed

Program Administrator

- Create and manage placement records
 - Maintain reference data needed by the system
 - Monitor application and approval workflow status
-

4. User Journey

Main Journey

1. A student signs in and browses internship opportunities.
 2. The student opens a placement detail page and reviews requirements.
 3. The student submits an application for a placement.
 4. The advisor reviews the application and approves or rejects it.
 5. The student tracks status changes and submits progress updates after approval.
-

5. Functional Requirements

FR-01 Opportunity Explorer

The system shall allow students to browse a list of internship placements with basic filtering by keyword, skill tag, or status.

FR-02 Placement Detail

The system shall allow students to view full placement details including title, organization, description, required skills, location, deadline, and availability status.

FR-03 Application Submission

The system shall allow an authenticated student to submit an application for a placement.

FR-04 Application Tracker

The system shall allow a student to view only their own applications and current statuses.

FR-05 Advisor Approval Queue

The system shall allow advisors to view pending applications that require review and approve or reject them.

FR-06 Progress Update Submission

The system shall allow students with approved placements to submit internship progress updates.

FR-07 Status History

The system shall record application and approval status changes with timestamps for auditability.

FR-08 Role-Based Access

The system shall enforce role-based access for students, advisors, and administrators.

FR-09 Placement Management

The system shall allow program administrators to create, update, publish, and archive placement records.

FR-10 Status Change Event

The system shall publish an `internship.status_changed` event when a relevant application or internship status changes.

FR-11 Skill Matching Support

The system should support placement recommendations based on student skill data when available, with a fallback to declared skill tags if AI-assisted matching is unavailable.

6. Non-Functional Requirements

NFR-01 Performance

The system should load the opportunity explorer and placement detail pages within a few seconds under normal student-project usage.

NFR-02 Security

The system must require authentication for non-public actions and must restrict data access by role and ownership.

NFR-03 Availability

The system should remain available on free-tier hosting for demo, testing, and moderate class-project usage.

NFR-04 Cost

The system must be deployable with a 0 THB infrastructure budget using free-tier services.

NFR-05 Maintainability

The codebase should be simple enough for 3-5 student developers to understand and extend within one semester.

NFR-06 Reliability

The system should prevent duplicate application creation and preserve consistent state across approval actions.

7. Business Rules

BR-01

A student may only view and manage their own applications and progress updates unless elevated access is explicitly granted to staff.

BR-02

An advisor approval or rejection decision must be recorded with timestamp and reviewer identity.

BR-03

A student cannot submit a progress update unless their internship application has been approved or marked active.

BR-04

The system should prevent duplicate active applications by the same student for the same placement unless the team intentionally supports re-application.

BR-05

Only administrators may create or edit placement records.

BR-06

Status changes that affect downstream systems must emit an `internship.status_changed` event.

8. Data Model

UserProfile

- id
- identity_ref
- name
- email
- role (student, advisor, admin)
- department
- skill_tags

Notes:

- This is a minimal application profile linked to identity data rather than a full identity store.

Placement

- id
- title
- organization_name
- description
- required_skills
- location
- application_deadline
- status
- created_by
- created_at
- updated_at

Application

- id
- student_id
- placement_id
- status
- statement
- submitted_at
- updated_at

Approval

- id
- application_id
- advisor_id
- decision
- comment
- decided_at

ProgressUpdate

- id
- application_id
- student_id
- week_number
- summary
- status
- submitted_at

Core Relationships

- One UserProfile can create many Application records as a student.
- One Placement can have many Application records.
- One Application can have zero or more Approval records, but only one final active decision for MVP.

- One Application can have many ProgressUpdate records after approval.

Data That Should Not Be Stored

- Student passwords if external auth is used
- Full identity provider records beyond necessary claims
- Sensitive private chat content unrelated to internship workflow

Sensitive Data

- Student personal identifiers
- Student email addresses
- Advisor decisions and comments

9. Architecture

Architecture Overview

The system will use a simple full-stack web architecture with Next.js for the frontend and server-side API layer, Supabase PostgreSQL for relational application data, Supabase Auth for authentication, Supabase Storage for optional document storage, and Vercel for deployment.

Architecture Diagram

Student/Advisor/Admin Browser -> Next.js Frontend -> Next.js API / Server Actions -> Supabase PostgreSQL

Next.js API / Server Actions -> Supabase Auth for sign-in and role-aware access

Next.js API / Server Actions -> Supabase Storage for optional file storage

Next.js API / Server Actions -> External identity claims / job board listing source / status event consumers

Components

Frontend

- Next.js web application
- Opportunity explorer, placement details, application tracker, advisor review queue, progress update form
- App routing, authenticated session handling, and server/client rendering where appropriate

Backend

- Next.js API routes or server-side handlers
- Endpoints for placements, applications, approvals, and progress updates
- Input validation, authorization checks, and event publishing

Database

- Supabase PostgreSQL for core relational MVP data
- Tables for user_profiles, placements, applications, approvals, and progress_updates
- Constraints to prevent duplicate student applications for the same placement where required

Authentication

- Supabase Auth will manage sign-in and session handling.
- Application roles such as student, advisor, and admin can be mapped through profile data and enforced in the application and database access layer.

Storage

- Supabase Storage for optional supporting files if the team later adds resume or report uploads
- Not required for the narrowest MVP if file upload is excluded

External Services

- Identity profile or skills claims source
- Job board listing source
- Downstream event consumers for internship.status_changed

10. Technology Stack

Layer | Technology | Reason

Frontend | Next.js | Combines frontend and backend patterns in one framework and is easier to manage for a student team

Backend | Next.js Server/API | Keeps API logic close to the application and reduces infrastructure complexity

Database | Supabase PostgreSQL | Familiar relational model with strong free-tier support for MVP data

Auth | Supabase Auth | Built-in authentication reduces setup overhead

Storage | Supabase Storage | Optional file storage integrated with the same platform

Hosting | Vercel | Straightforward deployment for Next.js projects

11. API / Interfaces

API-01 Placements

- GET /placements
- GET /placements/{id}

API-02 Applications

- POST /applications
- GET /applications/me
- PATCH /applications/{id}

API-03 Approvals

- POST /applications/{id}/approve

API-04 Progress Updates

- POST /progress-updates

API-05 Status Events

- Publish `internship.status_changed` when application or internship status changes
-

12. Security

Authentication

- All student, advisor, and administrator actions require authenticated access.
- Public browsing may be allowed only for placement listings if the team wants a simpler demo flow.

Authorization

- Students may access only their own applications and progress updates.
- Advisors may access only review-related records within their allowed scope.
- Administrators may manage placement records and limited operational data.

Data Protection

- Validate all inputs at the API boundary.
 - Minimize stored personal data.
 - Avoid storing secrets or credentials in source control.
 - Protect event payloads from exposing unnecessary personal information.
-

13. Error Handling

Expected Errors

- Invalid placement ID
- Missing required application fields
- Unauthorized access
- Forbidden role action
- Duplicate application submission
- Invalid status transition

Failure Scenarios

- Two approval actions are submitted at nearly the same time
- A student submits the same application twice
- Database write fails after business validation succeeds
- Identity or job-board integration is unavailable
- Event publishing fails after a status update

Mitigation for MVP:

- Use unique constraints where appropriate
 - Re-check status before updates
 - Return clear API error messages
 - Log failures for retry or manual follow-up
-

14. Deployment

Development

- Local Next.js development environment
- Shared Supabase project or local development database setup for testing
- Shared development database and seed data for demo workflows

Production

- Deploy the Next.js application on Vercel
 - Use Supabase PostgreSQL for application data
 - Use Supabase Storage only if file storage is included in scope
 - Configure environment variables for auth and integrations
-

15. Constraints

- Budget: 0 THB
 - Time: one semester
 - Team: 3-5 student developers
 - Free Tier: architecture must fit within free-tier service limits
-

16. Risks

Risk | Impact | Mitigation

Next.js and Supabase are new to some team members | Slower onboarding and uneven implementation quality | Build a small end-to-end prototype in the first sprint and standardize patterns early

Role and permission rules are implemented inconsistently | Users may access the wrong data | Define access rules early and test student, advisor, and admin cases explicitly

Duplicate or inconsistent approval actions | Incorrect internship status | Use validation, unique constraints, and status transition checks

Integration dependencies are unavailable | Demo flow may break | Mock identity, job board, and event consumers for MVP

Scope grows beyond semester limits | Incomplete delivery | Keep file uploads, advanced AI, and analytics as future work

17. Acceptance Criteria

MVP is complete when:

- [] Students can browse placements and open placement details
- [] Authenticated students can submit an application
- [] Students can view their own application statuses
- [] Advisors can approve or reject pending applications

- ☐ Approved students can submit progress updates
 - ☐ Role-based access is enforced for student, advisor, and admin actions
 - ☐ The system prevents or clearly handles duplicate applications
 - ☐ Status changes can trigger an `internship.status_changed` event
 - ☐ The system is deployable on free-tier infrastructure
-

18. Future Improvements

- Resume or document upload support using Supabase Storage
- Better placement recommendation quality with stronger AI matching
- Notification delivery by email or in-app alerts
- Analytics dashboard for departments and program administrators
- Employer-facing placement submission workflow